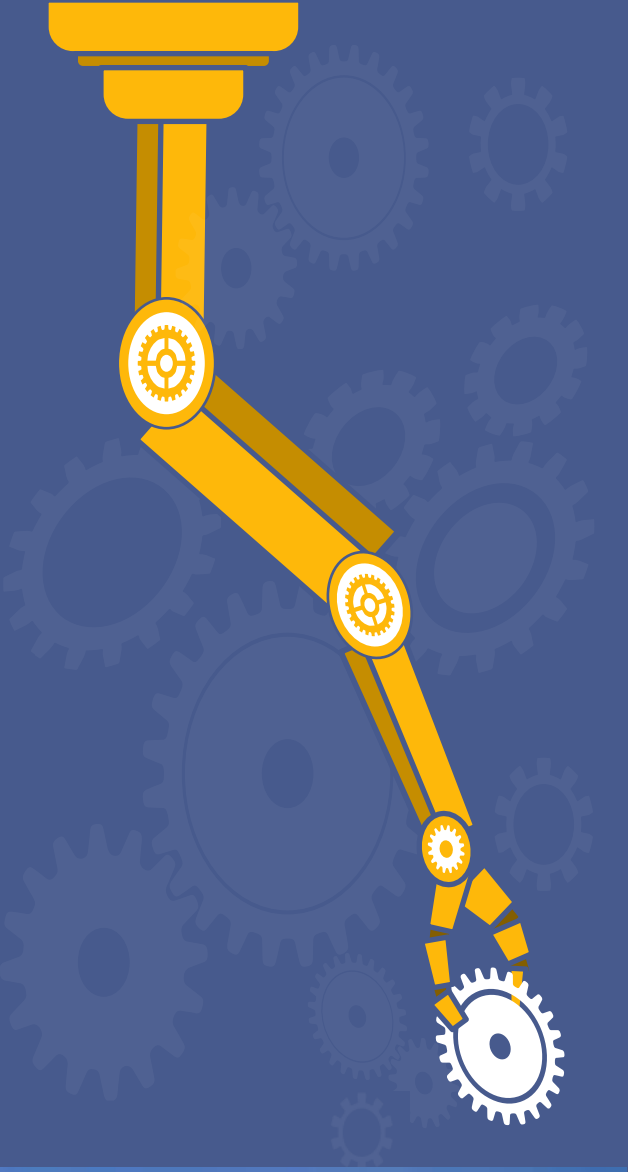


VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT, Vellore.

BCSE204L- Design and Analysis of Algorithms-

Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms

2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively

3. To synthesize efficient algorithms in various engineering design situations

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms

2. Demonstrate the major algorithm design paradigms.

3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.

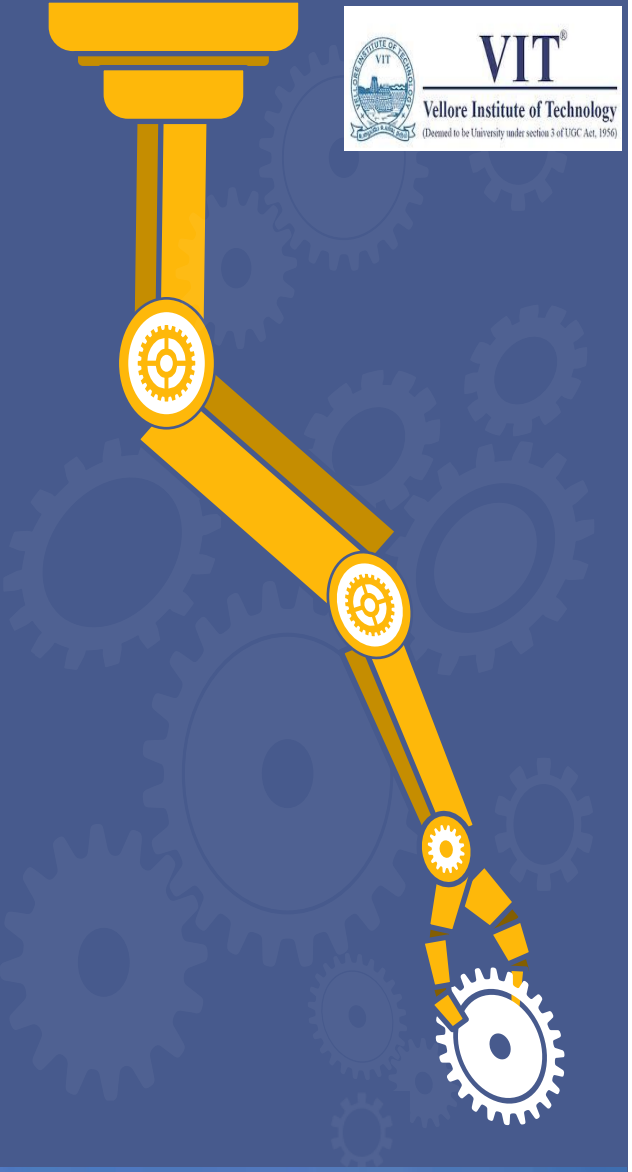
4. Articulating Randomized Algorithms.

5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	**Design Paradigms: Greedy, Divide and Conquer Techniques**	**6 hours**
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	**Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques**	**10 hours**
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	**String Matching Algorithms**	**5 hours**
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	**Graph Algorithms**	**6 hours**
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	**Geometric Algorithms**	**4 hours**
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	**Randomized algorithms**	**5 hours**
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	**Classes of Complexity and Approximation Algorithms**	**7 hours**
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	**Contemporary Issues**	**2 hours**



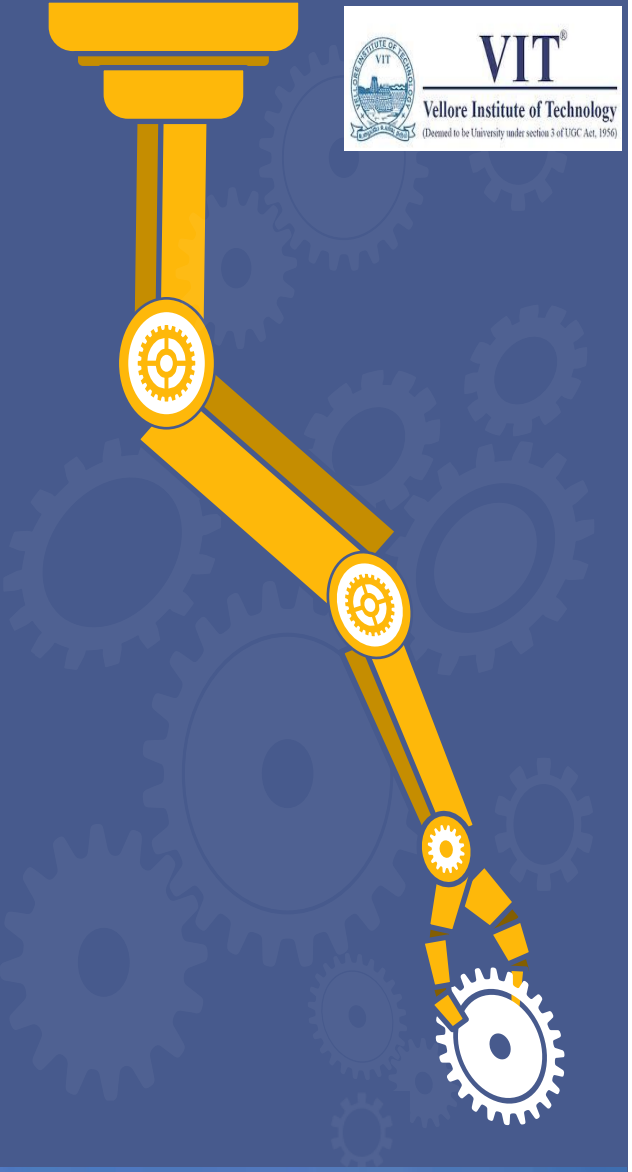
BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, ÉvaTardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press,1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, " Network Flows: Theory, Algorithms and Applications", Pearson Education, 2014.



Module 7 : Classes of Complexity and Approximation Algorithms (7 Hours)

- Class P- NP- Reducibility and NP- Completeness
- SAT, 3-SAT
- Independent Set
- Clique Decision Problem
- Approximation Algorithm
  - Vertex Cover
  - Set Cover
  - Travelling Salesman Problem

Deterministic vs Non-Deterministic - Recall

Deterministic

- Can determine the next step of the algorithm.
- Result of every operation is uniquely defined
- Can solve the problem in polynomial time

Non-Deterministic

- Cannot determine the next step of the algorithm.
- Algorithm contains operation whose outcomes are limited to specific set of possibilities
- Can solve the problem in Exponential time

Approximation Algorithms

- **Introduction**
  - Also called as Heuristic Algorithm
  - Way of approaching NP-Completeness for the optimization Problem and returns near optimal solutions.
- **Objective**
  - To Design an algorithm and come close as possible to the optimum value in reasonable amount of time(i.e at most Polynomial Time). Does not guarantee the best solution.
- **Example:**
  - In TSP , The optimization problem is to find the **shortest cycle**, and the approximation problem is to find a **short cycle**
  - **For Vertex Cover Problem,** The optimization problem is to find the **vertex cover with fewest vertices** , and the approximation problem is to find the vertex cover with few vertices

How to measure the quality of approximation Algorithm

- Accuracy Ratio/ Approximation Ratio

- For Example: Any Maximization Problem

Exact Solution

$r(S_a) = f(s^*)/ f(S_a)$ - cost of optimal solution is larger than the cost of approximate solution

- For Example: Any Minimization Problem

$r(S_a) = f(S_a)/f(s^*)$ - cost of approximate solution is larger than the cost of optimal solution

Approximation Solution

Approximation Algorithms- Definition

- Given any optimization problem "P", an algorithm "A" is said to be an approximation algorithm for "P", if for any given instance I, it returns an approximate solution, that is a feasible solution.
- P – Optimization Problem
- A - Approximation algorithm
- I- Instance of P
- A*(I)- Optimal value for the Instance I
- A(I) – Value for the instance I generated by A

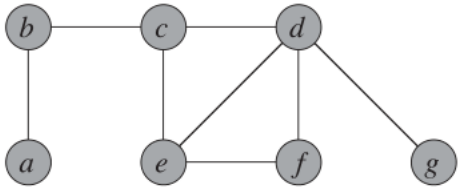
- Examples
  - Vertex Cover
  - TSP

Vertex Cover Problem

- A vertex cover of an undirected graph G={V,E} is a subset V'ε V such that if(u,v) is an edge of G, then either u ε V' or v ε V' or both. The size of the vertex cover is number of vertices on it.
- Objective : Finding a vertex cover of minimum size in a given undirected graph.
- The optimization problem is to find **vertex cover with fewest vertices**.
- The approximation problem is to find **vertex cover with few vertices.**

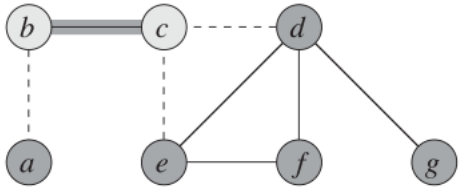
Vertex Cover Problem

7 vertices and 8 edges



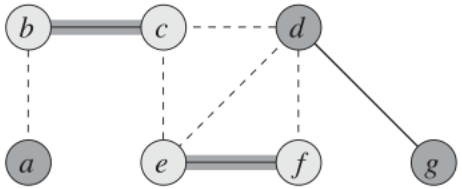
(a)

b, c first edge selected

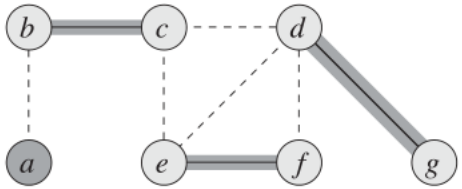


(b)

e, f edge selected

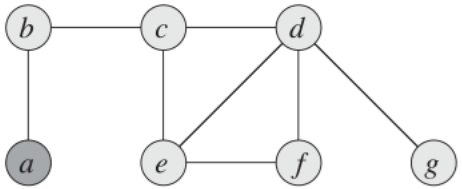


(c)



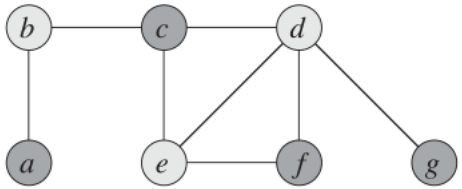
(d)

d, g edge selected



(e)

vertex cover produced by Approximate algorithm- Polynomial time



(f)

Optimal vertex cover for this problem- Exponential time

Vertex Cover Problem- Approximate Algorithm

```
APPROX-VERTEX-COVER(G)
1  C = ∅                                  ← Empty set
2  E' = G.E                               ← Copy of all edges
3  while E' ≠ ∅
4      let (u, v) be an arbitrary edge of E'
5      C = C ∪ {u, v}                     ← Add it end points to C
6      remove from E' every edge incident on either u or v
7  return C
```

C={b,c,e,f,d,g} – 6 vertices

Running time : O(V+E)

2-approximation algorithm.

- If the accuracy ratio is 2 we have a 2-approximation algorithm, it means
- Gives solutions that never cost more than twice that of optimal if it is a minimization problem.
- Never provide less than half the optimal value if it is a maximization problem.

Proof of Theorem

- **Theorem: APPROX -VERTEX -COVER is a polynomial-time 2-approximation algorithm.**
- C- is a set returned by APPROX -V ERTEX –COVER
- APPROX -VERTEX -COVER returns a vertex cover that is at most twice the size of an optimal cover
- A denote the set of edges APPROX -VERTEX -COVER picked.
- A={(b,c),(e, f),(d,g)}
- In order to cover the edges in A, **any vertex cover say example: Optimal Cover** denoted by C^* must include at least one endpoint of each edge in A.
- No two edges in A share an endpoint, since once an edge is picked, all other edges that are incident on its endpoints are deleted from E'.

Proof of Theorem

- Thus, no two edges in A are covered by the same vertex from C* , and we have the lower bound

  |C*|>=|A| on the size of the optimal vertex cover. -----eqn 1

- Each execution of line 4 picks an edge for which neither of its endpoints is already in C, yielding an upper bound (an exact upper bound, in fact) on the size of the vertex cover returned.

  |C|=2|A| ----- eqn 2

- Combining the two equations

  |C|=2|A|

  <=2|C*|

- Hence proved 2- approximation algorithm